



GUIA DE DESENVOLVIMENTO COM GIT



OBJETIVO

Padronizar o versionamento de código-fonte entre todos os trabalhadores, garantindo a colaboração eficiente entre todos, além de permitir a criação de fluxos automatizados de testes e deploy.

FLUXO PARA INICIAR O DESENVOLVIMENTO

Sempre inicie o trabalho isolando suas alterações da linha estável de produção em uma branch partindo como base a branch principal do projeto (normalmente “main”).

```
# Mude para a branch principal
git switch main

# Baixe as atualizações mais recentes da branch principal
git pull

# Crie a branch substituindo <nome-nova-branch> pelo nome correto
git switch -c <nome-nova-branch>

# Crie um commit vazio para limpar o HEAD
git commit --allow-empty -m "cmd: track"

# Envie a branch local para o GitLab com o mesmo nome e com track
git push -u origin <nome-nova-branch>
```

A intenção do primeiro envio ao GitLab ser com o parâmetro de track é para que no futuro só precise dos comandos “git pull” e “git push” para enviar e puxar as atualizações, sem precisar indicar novamente a origem e destino.

BRANCHES PERMANENTES NO PROJETO

Todo projeto sempre contém 3 branches fixas. Cada branch possui sua função no projeto e deve-se seguir as seguintes regras:

Branch dev

Essa branch é o “caldeirão” e integração.

Regra: onde todos os desenvolvedores juntam seus códigos. É instável por natureza por receber novas atualizações. É a primeira branch a receber o Merge Request e rodar o pipeline (MR).

Ação do Pipeline: roda testes pesados e gera imagens de snapshot (dev-latest).



Branch hml

Essa branch é para o ambiente de homologação. Roda um servidor, ou conjunto de servidores, para efetuar os testes da Entrega.

Regra: onde o PO e QA validam as funcionalidades integradas.

Ação do Pipeline: gera uma imagem candidata (ex.: v1.0.0-rc.1) e faz deploy automático no ambiente de homologação.

Branch main

Representa o ambiente de Produção.

Regra: ninguém faz commit direto nessa branch, devendo ser sempre um Merge Request (MR) vindo da branch hml.

Ação do Pipeline: gera a **Tag de Versão** (ex.: v1.0.0) e a **Imagem Docker Final**.

BRANCHES TEMPORÁRIAS NO PROJETO

As branches que possuem origem em uma issue, as criadas para desenvolvimento, são temporárias no projeto, sendo apagadas após o Merge Request final.

É possível subir um ambiente, em um servidor temporário, quando é adicionado comando “up_temporary_app 24h” no corpo de um commit enviado ao GitLab. Desta forma é possível efetuar alguns testes de integração em um servidor real. O servidor será mantido pela quantidade de horas indicadas no comando.

REGRAS PARA OS COMMITS

Os commits são parte fundamental de todo o histórico do desenvolvimento, por isso recebem atenção especial.

Efetue sempre **Commits Atômicos**, cada commit faz apenas uma coisa, como corrigir um erro ou adicionar uma função, e, preferencialmente, não devem quebrar testes já existentes.

Para garantir a boa comunicação com os commits, eles devem ser escritos em **português** e deve-se utilizar as regras do **Conventional Commits**. De forma resumida:

No título do commit indique o seu tipo, seguido de uma **descrição clara e no imperativo**. A descrição no título deve informar o que o commit faz quando aplicado, e não evite ultrapassar **50 caracteres**.

Os tipos disponíveis são:

- feat: nova funcionalidade
- fix: correção de bug



- chore: tarefas menores
- refactor: melhoria de código
- style: estilo e/ou formatação
- test: criação de testes
- docs: documentação
- perf: performance

Exemplos de títulos de commit:

- `git commit -m 'fix: impede e-mail vazio no cadastro do usuário'`
- `git commit -m 'feat: adiciona autenticação m2f'`

Em algumas situações pode ser necessário explicar algo relacionado ao commit, indicando algum detalhe técnico por exemplo. Nessas situações, mantenha o texto claro e objetivo e evite ultrapassar **72 caracteres** por linha.

Para adicionar o corpo a um commit basta executar o comando de commit sem fornecer o parâmetro “-m”. Ex.: “git commit”.

Conclusão de Issue

Todo commit que concluí uma issue deve ter:

1. Título do commit com o mesmo nome do título da issue.
2. Se há Breaking Changes (alterações que quebram alguma funcionalidade antiga) é obrigatório adicionar o sinal de exclamação no título do commit.
3. Corpo descrevendo o commit.
4. No corpo do commit deve-se adicionar o comando que fecha a issue: Closes #<ID da Issue>

Exemplo de um commit que concluí uma issue:

```
feat: Adiciona login com AMEI
```

```
Foi criada a parametrização com o AMEI para permitir o login unificado,  
tendo sido criada a autorização da aplicação junto ao Sebrae NA.
```

```
Closes #17
```

```
Chamado 2570
```

```
CMD: up_temporary_app 24h
```

```
BREAKING CHANGE: O login antigo não funciona. Os usuários devem criar  
conta com o AMEI. O sistema vinculará a conta AMEI com a antiga conta  
usando o e-mail como chave.
```



CRIAÇÃO DE MERGE REQUEST (MR) - INICIANDO A INTEGRAÇÃO

O Merge Request é a unidade central de colaboração, permitindo a revisão de código e discussão antes da integração final.

Antes de criar um MR, garanta que a sua branch está atualizada com a main. Faça um pull da main, efetue o merge da main na sua branch e resolva todos os conflitos localmente.

Depois de gerar o commit que efetua uma entrega (concluí uma issue), envie o push junto com a solicitação de MR.

```
# [-o <string> | --push-option=<string>]  
git push -o merge_request.create -o merge_request.target=my-target-branch -o  
merge_request.squash
```

Revisão e Discussão

O desenvolvedor e o líder do projeto precisam conversar sempre que um código não passar na validação, definindo qual será o próximo passo a seguir.

Caso o MR mostre conflitos no momento de concluir o merge na interface, esses conflitos devem ser resolvidos na presença dos autores do MR e com o líder do projeto.

Caso os revisores solicitarem mudanças, não envie um novo MR. Simplesmente faça as alterações e envie os commits para a mesma branch do MR, pois ele será atualizado automaticamente.

CONTADOR DE VERSÃO

A versão do produto deve seguir o padrão do [Versionamento Semântico](#).

Desta forma a versão é dividida em três partes numéricas, separadas por ponto, que são incrementadas conforme o tipo da mudança, seguindo o padrão Major.Minor.Patch (Maior.Menor.Correção).

Uma mudança de Correção indica a eliminação de algum erro, não interferindo no funcionamento correto da Entrega. Uma mudança Menor indica a adição de funcionalidades, sem corromper o que já estava funcionando. Uma mudança Maior indica uma grande alteração, podendo influenciar em algo que já funcionava, removendo a funcionalidade, por exemplo.



Histórico de Revisões:

Data	Versão	Descrição
25/08/2026	1	Implantação do documento.